OpenAPI, REST

PA165 Java Enterprise Applications (2026)

Martin Kuba <makub@ics.muni.cz>

Outline

- theory
  - API calls
    - RPC, IDL
    - RMI
    - SOAP/WSDL
    - REST/OpenAPI
  - HTTP
  - handling HTTP in Java - Servlets, RestControllers
  - data formats - JSON, YAML, CBOR
- OpenAPI
  - info, tags, components, paths
  - data types, parameters
  - inheritance
- Spring Boot, Spring MVC, SpringDoc

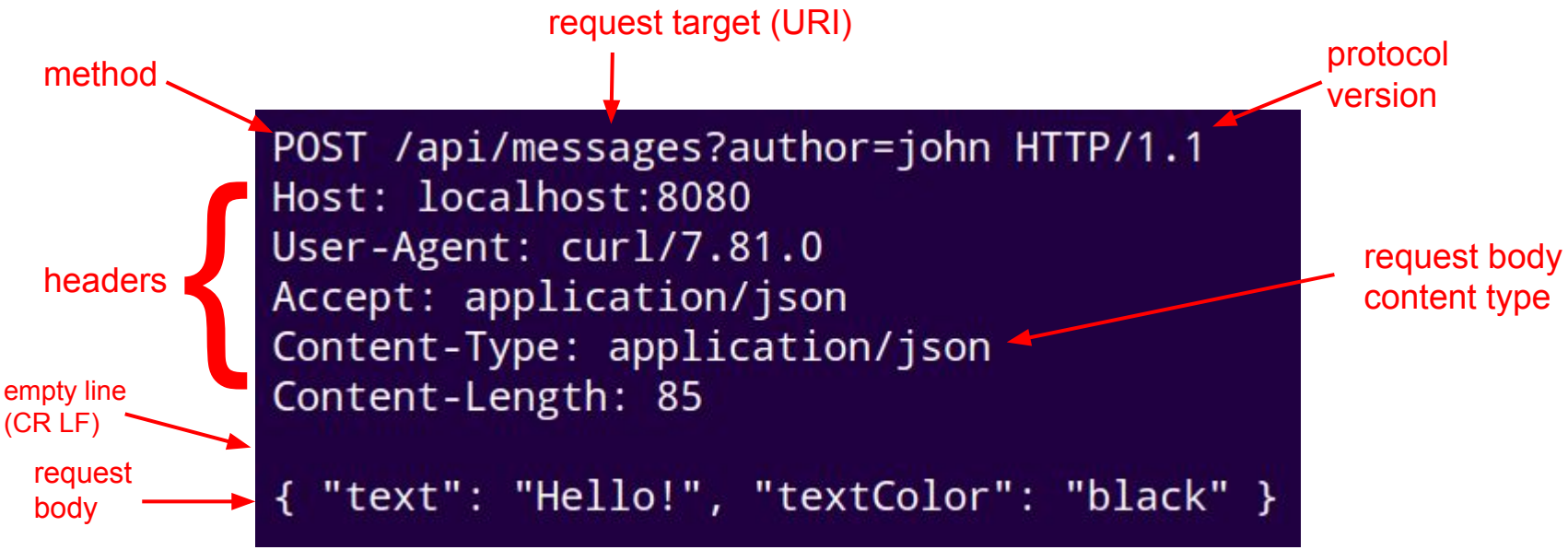
Remote API Calls

- **RPC - Remote Procedure Call**
  - introduced in early 1980s
  - enables calling remote procedures locally
- **IDL - Interface Description Language**
  - client **stub** or server **skeleton** generated in a given programming language from an IDL document
- **RMI - Remote Method Invocation**
  - introduced in early 1990s
  - distributed object-oriented systems
  - examples: CORBA, Java RMI, Microsoft DCOM
- **SOAP - Simple Object Access Protocol**
  - introduced in the year 2000
  - RPC protocol based on XML messages and HTTP
  - IDL is WSDL (Web Service Description Language)

REST - Representational State Transfer

- introduced in the year 2000 by HTTP's author Roy Fielding
- architectural style inspired by the architecture of WWW
- **resources** are identified by **URIs** (URLs or URNs)
- the same resource can be presented in various formats - JSON, XML, YAML, CBOR, …
- uses HTTP methods GET, PUT, DELETE, POST and PATCH for manipulating resources
- **verbs** (GET, PUT,...) manipulate **nouns** (resources)
- not every service using HTTP and JSON is RESTful
  - **RESTful**: `GET /message/1` (few verbs, many nouns)
  - **RPC style**: `getMessage(1)` (many verbs, many nouns)

HTTP - Hypertext Transfer Protocol





HTTP Methods

GET	requests a representation of the specified resource
HEAD	asks for a response identical to a GET request, but without the response body
POST	submits an entity to the specified resource, often causing a change in state or side effects on the server
PUT	replaces all current representations of the target resource with the request's payload
DELETE	deletes the specified resource
OPTIONS	describes the communication options for the target resource
PATCH	applies partial modifications to a resource

HTTP Status Codes

- 1XX - informational
  - 101 Switching Protocols
- 2XX - success
  - **200 OK**
  - 201 Created
- 3XX - redirection
  - 301 Moved Permanently
  - 302 Moved Temporarily
- 4XX - client error
  - 401 Unauthorized
  - **404 Not Found**
- 5XX - server errors
  - 500 Internal Server Error
  - 503 Service Unavailable

Content-Type / Media type

- originally MIME (Multipurpose Internet Mail Extensions) for specifying types of email attachments
- reused by HTTP for types of bodies of requests and responses
- format type "/" [tree "."] subtype ["+" suffix]* [";" parameter]
- types are registered in the [IANA Media Type Registry]

Examples:

text/html; charset=utf-8	HTML document encoded in UTF-8
image/png	image in PNG format
application/x-www-form-urlencoded	request from a submitted HTML form, format is the same as URL parameters, i.e. param1=value1&param2=value2
application/vnd.ms-excel	Microsoft Excel file (“vnd” - vendor tree)
application/json	general JSON text
application/at+jwt	Access Token in JSON Web Token format

Java Servlets

low-level handling of HTTP protocol in Java

```java
import jakarta.servlet.*;
import jakarta.servlet.http.*;

no usages
@WebServlet(urlPatterns = "/my/*")
public class MyServlet extends HttpServlet {

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        String method = req.getMethod();
        String contentType = req.getContentType();
        String path = req.getRequestURI();
        Map<String, String[]> parameterMap = req.getParameterMap();
        ServletInputStream inputStream = req.getInputStream();
        // read from inputStream

        resp.setContentType("text/html;charset=utf-8");
        resp.setStatus(200);
        resp.setHeader( name: "X-My-Header", value: "whatever");
        Cookie cookie = new Cookie( name: "mycookie",  value: "somevalue");
        cookie.setSecure(true); cookie.setHttpOnly(true);
        resp.addCookie(cookie);
        ServletOutputStream outputStream = resp.getOutputStream();
        // write to outputStream
    }
}
```

Spring MVC RestControllers

- high-level handling of HTTP
- two possible ways of returning values:
  a. return an object to be serialized if OK, or throw a ResponseStatusException for other HTTP statuses
  b. return parametrized ResponseEntity<>

```java
@RestController
@RequestMapping(path = "/api", produces = MediaType.APPLICATION_JSON_VALUE)
public class MyRestController {

    @GetMapping(path = "/message1/{id}")
    public ChatMessage getMessage1(@PathVariable String id) {
        ChatMessage m = getMessageById(id);
        if (m != null) {
            return m;
        } else {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "message with id=" + id + " not found");
        }
    }

    @GetMapping(path = "/message2")
    public ResponseEntity<ChatMessage> getMessage2(@RequestParam String id) {
        ChatMessage m = getMessageById(id);
        return new ResponseEntity<>(m, m != null ? HttpStatus.OK : HttpStatus.NOT_FOUND);
    }
```

JSON - JavaScript Object Notation

- lightweight data-interchange format, UTF-8 encoded text
- based on object syntax in JavaScript
- composition of hash tables, lists, and literals for strings, numbers, true, false and null
- no comments, strings are always in "double quotes"
- can describe only tree-like structures, no cycles
- **JSON Schema** can be used to describe data structures

```
{
  "key1": "some string",
  "key2": 123456,
  "key3": true,
  "key4": false,
  "key5": null,
  "key6": [ "one", 2, true, false, null, [], {} ],
  "key7": {
    "subkey": "multi\nline"
  }
}
```

YAML - Yet Another Markup Language

- officially “YAML Ain't Markup Language”
- superset of JSON - every JSON document is a YAML document
- more suitable for humans to write and read than JSON
- cyclic structures with anchors and references
- allows comments
- strings in " ", ' ' or without
- multiline strings
- folded strings
- https://quickref.me/yaml

```yaml
# this is a comment
key1: &anchor1 "some string"
key2:
  - "one"
  - 'two'
  - three
key3: [ "one", 'two', three ]
key4:
  subkey1: "A"
  subkey2: "B"
key5: { subkey3: "C", "subkey4": 'D' }
key6: |
  this is
  multi line text
key7: >
  this is folded
  single line text
key8: *anchor1
```

CBOR - Concise Binary Object Representation

- specification in RFC 8949
- binary data serialization format loosely based on JSON
- used in EU COVID certificate passes or Czech eDoklady
- smallest number of bytes - fits into QR codes
- network (big-endian) byte order
  - x86 CPUs are little-endian
  - ARM CPUs are little-endian
  - Java is big-endian, was developed for SPARC CPUs

OpenAPI

- IDL for HTTP-based APIs
- see https://github.com/OAI/OpenAPI-Specification
- the tool **OpenAPI Generator** can generate client stubs and server skeletons in many programming languages
- see https://github.com/OpenAPITools/openapi-generator
- OpenAPI Generator is an open-source fork of Swagger Codegen
- OpenAPI history
  - originally named Swagger in 2010
  - renamed to OpenAPI in 2016
  - version 3.0.0 released in 2017 - now used in Generator
  - version 3.1.0 released in 2021 - support still in beta
  - version 3.2.0 released in 2025 - not supported yet



OpenAPI document

- written in JSON or YAML
- the recommended name is **openapi.yaml** or openapi.json
- describes an API in terms of operations, their requests and responses

openapi.yaml

```yaml
openapi: 3.0.1
info:
  title: Pa165 Seminar OpenAPI Service
  description: |
    Hand-made OpenAPI document.
  version: 1.0.0
servers:
  - url: "http://localhost:8080"
tags:
  - name: MyService
    description: Tags are used for Java interface names
components:
  schemas: {}
paths:
  /api/hello:
    get:
      tags:
        - MyService
      summary: Returns "Hello World!"
      operationId: hello
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                type: object
                title: HelloResponse
                properties:
                  message:
                    type: string
```

version of the OpenAPI Specification used

version of the described API

base URL prepended to all paths

components is the place for definitions shared by many operations

URL path

HTTP method (get, post, put, …)

HTTP status code

Content-Type of response

properties of the returned JSON object

API-first or Code-first?

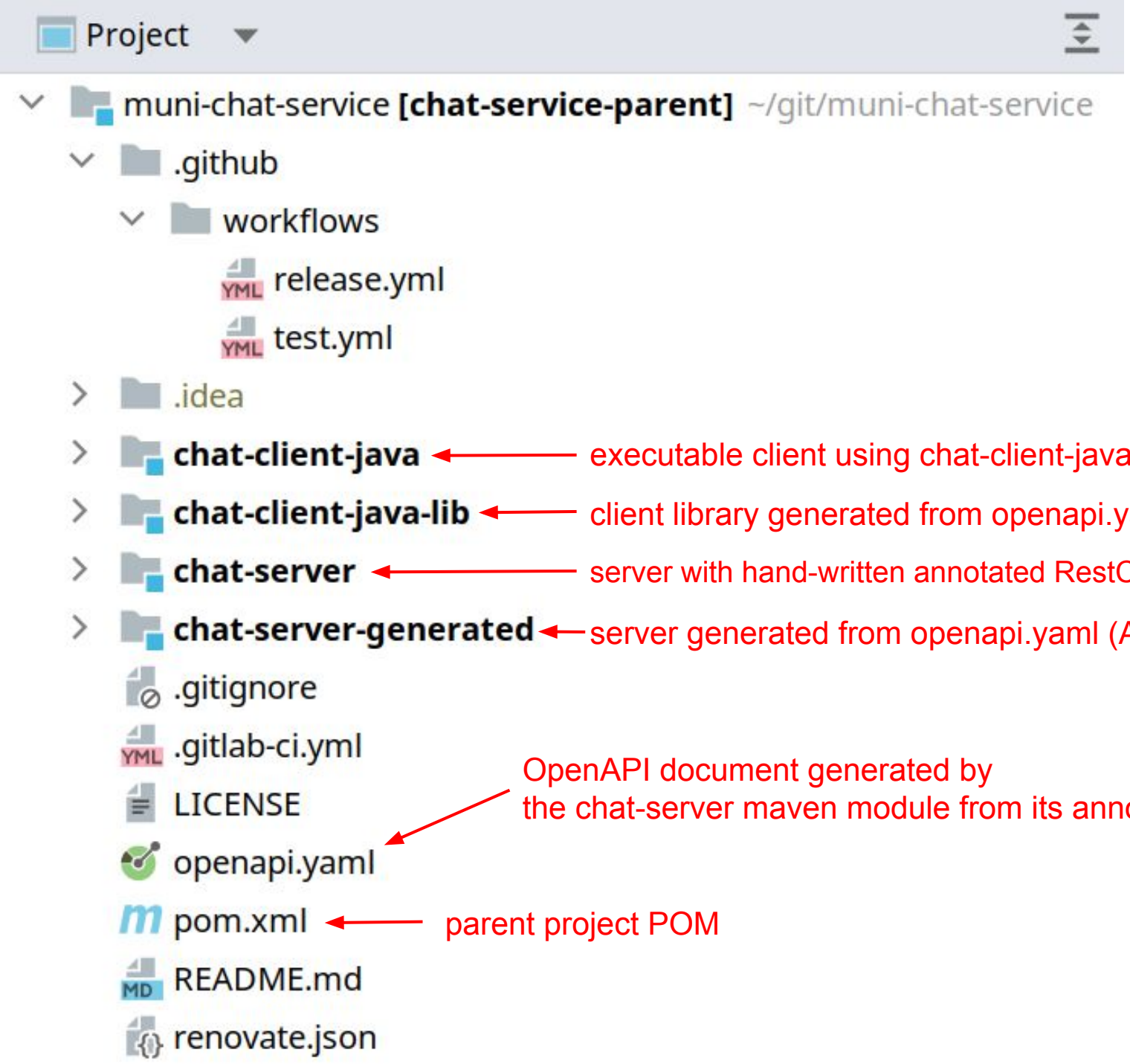
- two possible ways:
  - **API-first**
    i. write **openapi.yaml**
    ii. generate both client and server from the openapi.yaml
  - **Code-first**
    i. write Java **code** for a server
    ii. annotate its code with *SpringDoc OpenAPI* annotations
    iii. generate openapi.yaml from the code
    iv. generate client from the openapi.yaml
- API-first advantages
  - better control over openapi.yaml, e.g. shared definitions of parameters possible
- Code-first advantages
  - better control over code, e.g. can add @CrossOrigin for enabling JavaScript clients

Show me the code!

https://gitlab.fi.muni.cz/makub/muni-chat-service

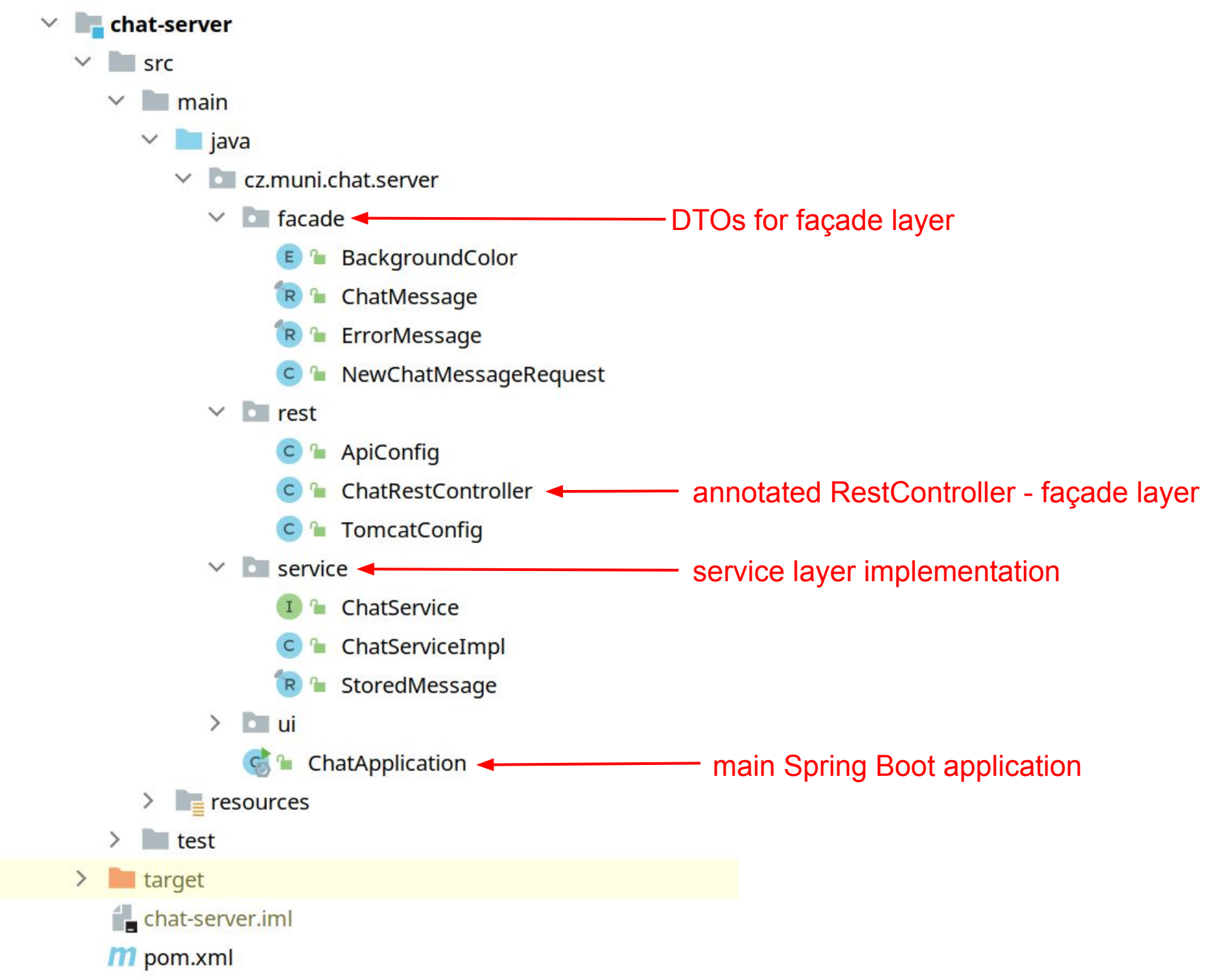
example of everything you need for Milestone 1:

- multi-module Maven project
- Spring Boot application
- Spring MVC RestController
- application with multiple layers (façade, service)
- OpenAPI - both code-first and API-first examples
- unit and integration tests


Project
muni-chat-service [chat-service-parent] ~/git/muni-chat-service
.github
workflows
release.yml
test.yml
.idea
chat-client-java
executable client using chat-client-java-lib library to call server
chat-client-java-lib
client library generated from openapi.yaml
chat-server
server with hand-written annotated RestController (code-first example)
chat-server-generated
server generated from openapi.yaml (API-first example)
.gitignore
.gitlab-ci.yml
LICENSE
OpenAPI document generated by
the chat-server maven module from its annotated Java code
openapi.yaml
pom.xml
parent project POM
README.md
renovate.json

chat-server module - code-first approach

- separate facade and service layers
  - **façade layer** uses Data Transfer Objects (DTOs)
  - **service layer** uses entities
- generates openapi.yaml
  - SpringDoc OpenAPI annotations in code
  - dependency `springdoc-openapi-starter-webmvc-ui` for generating openapi.yaml on the fly
  - downloading it using `springdoc-openapi-maven-plugin` during Maven integration tests phase
- Maven tests
  - unit test run by “mvn test”
  - integration tests run by “mvn verify”
  - “mvn install” includes both test and verify phases


chat-server
src
main
java
cz.muni.chat.server
facade
DTOs for façade layer
BackgroundColor
ChatMessage
ErrorMessage
NewChatMessageRequest
rest
ApiConfig
ChatRestController
annotated RestController - façade layer
TomcatConfig
service
service layer implementation
ChatService
ChatServiceImpl
StoredMessage
ui
ChatApplication
main Spring Boot application
resources
test
target
chat-server.iml
pom.xml

Façade vs. Service Layer

- **Façade layer**
  - is exposed to the outside world
  - uses **DTOs** (Data Transfer Objects)
  - incoming DTOs are big, represent all data in a request
  - outgoing DTOs expose only what is needed
- **Service layer**
  - is internal
  - uses **entities**
  - entities represent data as stored internally
  - entities may contain internal properties that must not be exposed to the outside
- example:
  - incoming: NewChatMessageRequest → StoredMessage
  - outgoing: StoredMessage → ChatMessage
  - entity StoredMessage contains property secretData that is not present in the DTO ChatMessage

Entity in the Service Layer

```java
package cz.muni.chat.server.service;

import java.time.ZonedDateTime;

/**
 * Represents a message in the service layer.
 */
 Martin Kuba
public record StoredMessage(String id,
                            ZonedDateTime timestamp,
                            String text,
                            String author,
                            String textColor,
                            String backgroundColor,
                            Object secretData) {
}
```

internal data that must not be exposed

DTO (without OpenAPI annotations)

```
import jakarta.validation.constraints.NotBlank;
import java.time.ZonedDateTime;

public record ChatMessage(@NotBlank
                          String id,
                          @NotBlank
                          ZonedDateTime timestamp,
                          @NotBlank
                          String text,
                          String author,
                          String textColor,
                          BackgroundColor backgroundColor) {

    public static ChatMessage fromStoredMessage(StoredMessage m) {
        return new ChatMessage(m.id(), m.timestamp(), m.text(),
                m.author(), m.textColor(), BackgroundColor.fromValue(m.backgroundColor()));
    }
}
```

secret internal data
are not included!

DTO (with OpenAPI annotations)

```java
@Schema(title = "chat message",
        description = "represents a message in a chat"
)
public record ChatMessage(@NotBlank
                          @Schema(description = "uuid", example = "123e4567-e89b-12d3-a456-426614174000")
                          String id,
                          @NotBlank
                          @Schema(type = "string", format = "date-time",
                                  description = "time of creation",
                                  example = "2022-12-22T12:04:04.493908908+01:00")
                          ZonedDateTime timestamp,
                          @NotBlank
                          @Schema(description = "text of message", example = "Hello! \uD83D\uDE00")
                          String text,
                          @Schema(description = "author of message", example = "John")
                          String author,
                          @Schema(description = "HTML color name or RGB hex code", example = "black")
                          String textColor,
                          BackgroundColor backgroundColor) {

    public static ChatMessage fromStoredMessage(StoredMessage m) {
        return new ChatMessage(m.id(), m.timestamp(), m.text(),
                m.author(), m.textColor(), BackgroundColor.fromValue(m.backgroundColor()));
    }
}
```

@Schema annotations adding metadata for OpenAPI description

Generated Schema

```java
@Schema(title = "chat message",
        description = "represents a message in a chat
)
public record ChatMessage(@NotBlank
                          @Schema(description = "uuid
                          String id,
                          @NotBlank
                          @Schema(type = "string", fo
                                  description = "time
                                  example = "2022-12-
                          ZonedDateTime timestamp,
                          @NotBlank
                          @Schema(description = "text
                          String text,
                          @Schema(description = "auth
                          String author,
                          @Schema(description = "HTML
                          String textColor,
                          BackgroundColor backgroundC


    public static ChatMessage fromStoredMessage(Store
        return new ChatMessage(m.id(), m.timestamp(),
                m.author(), m.textColor(), Background
    }
}
```

```yaml
components:
  schemas:
    ErrorMessage: <4 keys>
    BackgroundColorEnum: <4 keys>
    NewChatMe…ageRequest: <4 keys>
    ChatMessage:
      title: chat message
      required:
      - id
      - text
      - timestamp
      type: object
      properties:
        id:
          type: string
          description: uuid
          example: 123e4567-e89b-12d3-a456-426614174000
        timestamp:
          type: string
          description: time of creation
          format: date-time
          example: 2022-12-22T12:04:04.493908908+01:00
        text:
          type: string
          description: text of message
          example: Hello! 😀
        author:
          type: string
          description: author of message
          example: John
        textColor:
          type: string
          description: HTML color name or RGB hex code
          example: black
        backgroundColor:
          $ref: '#/components/schemas/BackgroundColorEnum'
      description: represents a message in a chat
```

OpenAPI Primitive Data Types

OpenAPI schema	Java type
{ type: string }	java.lang.String
{ type: boolean }	java.lang.Boolean
{ type: integer }	java.lang.Integer
{ type: integer, format: int32 }	java.lang.Integer
{ type: integer, format: int64 }	java.lang.Long
{ type: number }	java.math.BigDecimal
{ type: number, format: double }	java.lang.Double
{ type: number, format: float }	java.lang.Float
{ type: string, format: byte }	byte[]
{ type: string, format: binary }	java.io.File
{ type: string, format: date }	java.time.LocalDate
{ type: string, format: date-time }	java.time.OffsetDateTime
{ type: string, format: uuid }	java.util.UUID

OpenAPI Complex Data Types

OpenAPI schema	Java type
{ type: array, items: { type: … } }	java.util.List<>
{ type: object }	java.lang.Object
{ type: object, properties: { … } }	generated java class with setters and getters
{ type: object, additionalProperties: { type: object } }	java.util.Map<String,Object>
{ type: object, properties: { … }, additionalProperties: true }	java.util.Map<String,Object> with setters and getters
{ type: array, uniqueItems: true, items: { type: string } }	java.util.Set<>

null in OpenAPI

OpenAPI 3.0	OpenAPI 3.1	Java type	Python type
{ type: integer, nullable: true }	type: ["integer", "null"]	JsonNullable<Integer>	(int, type(None))

- nullable in OpenAPI 3.0 is by default false
- in Java, forgetting to set `nullable: true` makes no problem
- in Python, a property without `nullable: true` cannot have value None
- JsonNullable has 3 choices - value, null or undefined

```
public static final String JSON_PROPERTY_HEIGHT = "height";
private JsonNullable<Integer> height = JsonNullable.<Integer>undefined();

@jakarta.annotation.Nullable
@JsonIgnore
public Integer getHeight() { return height.orElse( other: null); }

@JsonProperty(value = JSON_PROPERTY_HEIGHT, required = false)
@JsonInclude(value = JsonInclude.Include.USE_DEFAULTS)
public JsonNullable<Integer> getHeight_JsonNullable() { return height; }
```

Temporary warning about versions

- the situation in March 2026:
- Jackson is a library for JSON handling
- Spring Boot 3 (end-of-life June 2026) uses Jackson 2
- Spring Boot 4 (released November 2025) uses Jackson 3
- JsonNullable is a Jackson plugin
  - Jackson 3 compatibility in version 0.2.10 (released today 😢)
- OpenAPI Generator 7.21 supports
  - useSpringBoot4=true useJackson3=false for spring generator
  - openApiNullable=true for java generator and native library
- thus
  - the demo app at https://gitlab.fi.muni.cz/makub/muni-chat-service is updated to Spring Boot 4 without JsonNullable
  - the seminar app at https://gitlab.fi.muni.cz/pa165/seminar-open-api has branches spring_boot_3 and spring_boot_4 for both

Enumeration types in OpenAPI

the easy way - use allowableValues in @Schema:

```
@Schema(description = "HTML color name or RGB hex code",
        example = "black",
        allowableValues = {"black", "blue", "darkgrey"}
)
private String textColor = "black";
```



```
properties:
  text: <3 keys>
  textColor:
    type: string
    description: HTML color name or RGB hex code
    example: black
    enum:
    - black
    - blue
    - darkgrey
```

Enumeration types in OpenAPI (2)

the harder way - define an enum class

```
@Schema(
        name = "BackgroundColorEnum",
        title = "background color",
        description = "enumeration of allowed background colors",
        enumAsRef = true
)
public enum BackgroundColor {
    // available values
    LIGHTGRAY( value: "lightgray"),
    WHITE( value: "white"),
    AQUAMARINE( value: "aquamarine"),
    LIGHTYELLOW( value: "lightyellow"),
    LIGHTBLUE( value: "lightblue"),
    BISQUE( value: "#ffe4c4");

    private final String value;

    BackgroundColor(String value) { this.value = value; }
```

tells generator to define a schema in /components/schemas

```
properties:
  backgroundColor:
    $ref: '#/components/schemas/BackgroundColorEnum'
```

Inheritance in OpenAPI - API-first

```yaml
Organism:
  type: object
  properties:
    latinName: { type: string }
    typeName: { type: string }
  discriminator:
    propertyName: typeName
Animal:
  type: object
  allOf:
    - $ref: '#/components/schemas/Organism'
  properties:
    numberOfLegs: { type: integer }
  discriminator:
    propertyName: typeName
Pet:
  type: object
  allOf:
    - $ref: '#/components/schemas/Animal'
  properties:
    name: { type: string }
  discriminator:
    propertyName: typeName
```

discriminator property value must contain type name, i.e. Organism, Animal, or Pet

allOf inherits all properties from the referenced type

Inheritance in OpenAPI - code-first

```java
@Schema(
        discriminatorProperty = "typeName",
        subTypes = Animal.class,
        discriminatorMapping = {
                @DiscriminatorMapping(value = "cz.muni.chat.server.facade.Animal", schema = Animal.class),
                @DiscriminatorMapping(value = "cz.muni.chat.server.facade.Pet", schema = Pet.class)
        }
)
public class Organism {

    public String getTypeName() {
        return this.getClass().getTypeName();
    }
```

returns full class name

```java
@Schema( discriminatorProperty = "typeName", subTypes = Pet.class)
public class Animal extends Organism {
    int numberOfLegs;
```

```java
@Schema
public class Pet extends Animal {
    String name;

    public String getName() { return name; }

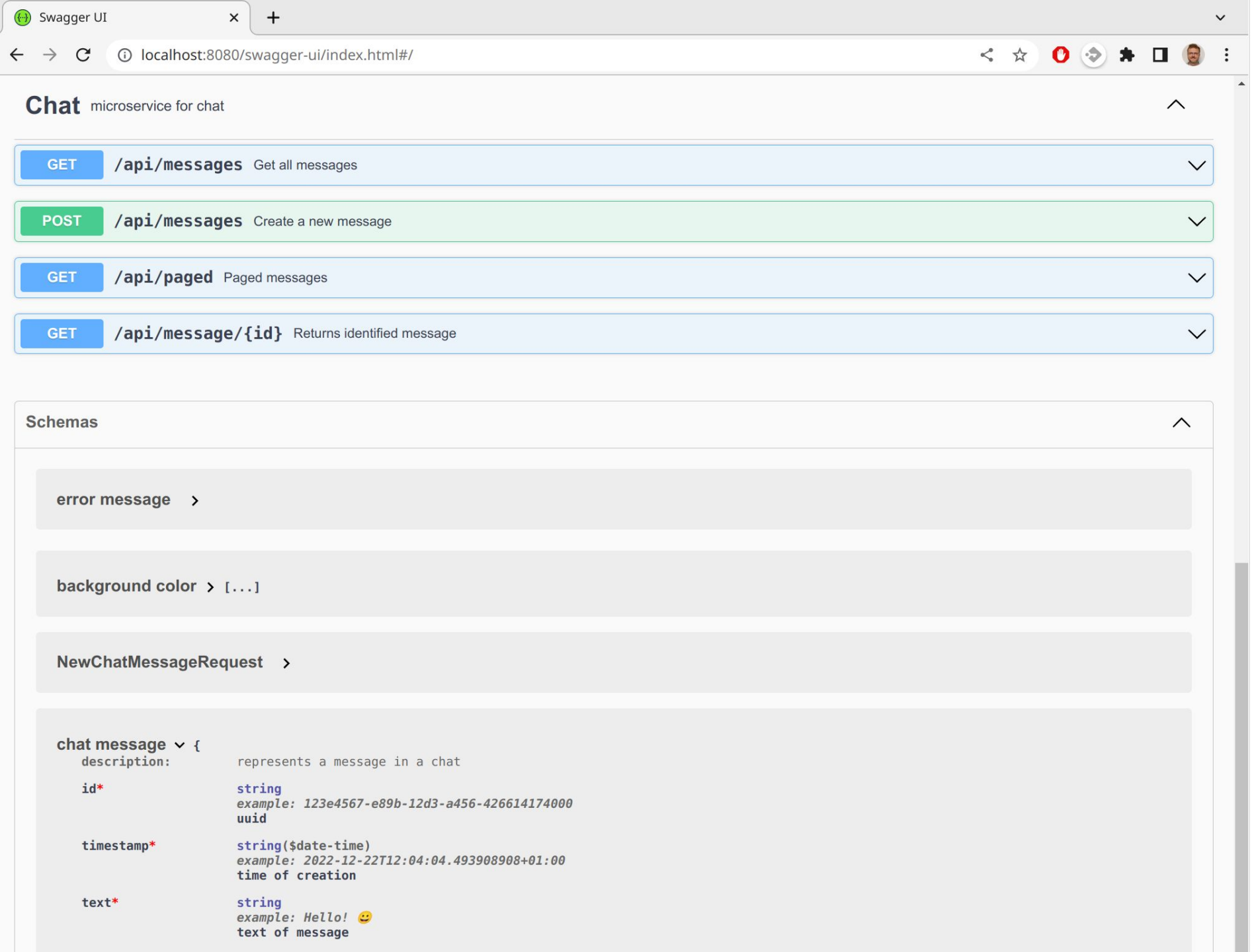
    public void setName(String name) { this.name = name; }
}
```

Operation - code-first

```java
    /**
     * REST method returning all messages.
     */
    @Operation( // metadata for inclusion into OpenAPI document
            summary = "Get all messages",
            description = """
                    Returns an array of objects representing chat messages, ordered from the newest to the oldest.
                    Each message must have a **text** and **timestamp**, and optionally may have an **author**,
                    a **text color** and a **background color**.
                    It is possible to use [MarkDown](https://www.markdownguide.org/) in descriptions.
                    """)
    @GetMapping(path = "/messages") // URL mapping of this operation
    @CrossOrigin(origins = "*") // CORS headers needed for JavaScript clients
    public List<ChatMessage> getAllMessages(HttpServletRequest req) {
        log.info("{} {} called from {}", req.getMethod(), req.getRequestURI(), req.getRemoteHost());
        // get messages from the service and convert them into DTOs that get serialized into JSON
        return chatService.getAllChatMessages().stream().map(ChatMessage::fromStoredMessage).toList();
    }
```



```yaml
paths:
  /api/messages:
    get:
      tags:
      - Chat
      summary: Get all messages
      description: |
        Returns an array of objects representing chat messages, ordered from the newest to the oldest.
        Each message must have a **text** and **timestamp**, and optionally may have an **author**,
        a **text color** and a **background color**.
        It is possible to use [MarkDown](https://www.markdownguide.org/) in descriptions.
      operationId: getAllMessages
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/ChatMessage'
```


Swagger UI
localhost:8080/swagger-ui/index.html#/
Chat microservice for chat
GET /api/messages Get all messages
POST /api/messages Create a new message
GET /api/paged Paged messages
GET /api/message/{id} Returns identified message
Schemas
error message
background color [...]
NewChatMessageRequest
chat message {
description: represents a message in a chat
id* string
example: 123e4567-e89b-12d3-a456-426614174000
uuid
timestamp* string($date-time)
example: 2022-12-22T12:04:04.493908908+01:00
time of creation
text* string
example: Hello!
text of message

Chat microservice for chat

GET /api/messages Get all messages

Returns an array of objects representing chat messages, ordered from the newest to the oldest. Each message must have a **text** and **timestamp**, and optionally may have an **author**, a **text color** and a **background color**. It is possible to use MarkDown in descriptions.

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/messages' \
  -H 'accept: application/json'
```

Request URL

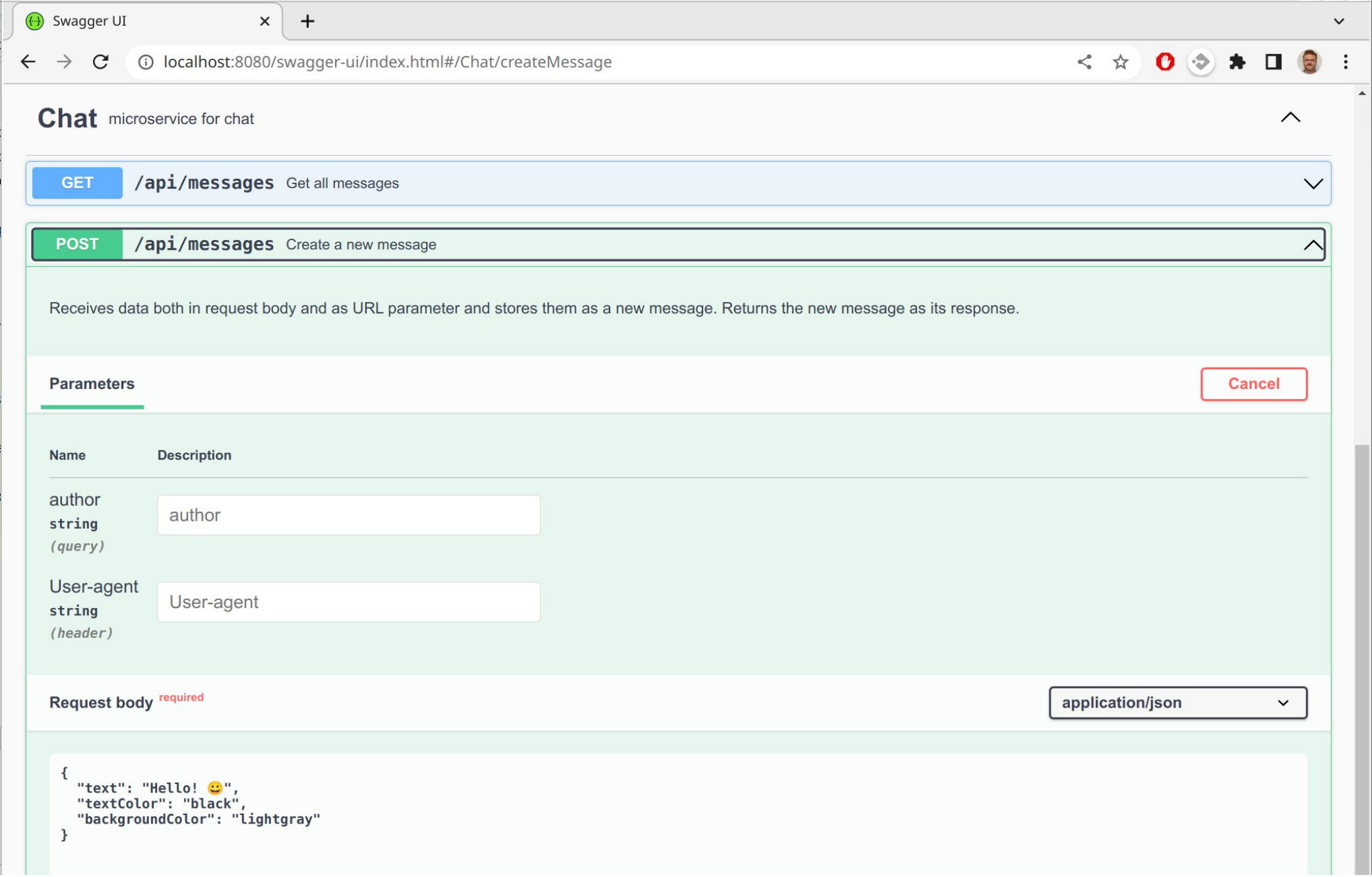
```
http://localhost:8080/api/messages
```

Server response

Code	Details
200	**Response body**

```
[
  {
    "id": "1030aac7-eee1-4431-8cb6-32ad44039533",
    "timestamp": "2023-02-03T14:30:11.7965308+01:00",
    "text": "first message",
    "author": "ⓘ system",
    "textColor": "black",
    "backgroundColor": "lightblue"
  }
]
```

Download


Swagger UI
localhost:8080/swagger-ui/index.html#/Chat/createMessage
Chat microservice for chat
GET /api/messages Get all messages
POST /api/messages Create a new message
Receives data both in request body and as URL parameter and stores them as a new message. Returns the new message as its response.
Parameters
Cancel
Name
Description
author
string
(query)
author
User-agent
string
(header)
User-agent
Request body required
application/json
{
"text": "Hello! 😀",
"textColor": "black",
"backgroundColor": "lightgray"
}

```yaml
post:
  tags:
  - Chat
  summary: Create a new message
  description: Receives … response.
  operationId: createMessage
  parameters:
  - name: author
    in: query
    required: false
    schema:
      type: string
  - name: User-agent
    in: header
    required: false
    schema:
      type: string
  requestBody:
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/NewChatMessageRequest'
    required: true
  responses:
    "201":
      $ref: '#/components/responses/SingleMessageResponse'
    "400":
      description: input data were not correct
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorMessage'
```

operation changes server state, hence POST HTTP method

input data in parameters

this parameter is in URL as ?author=<author>

this parameter is in HTTP header “User-agent”

HTTP request body with input data

input DTO

HTTP status code

responses can be defined in components too

```yaml
components:
  schemas:
    ErrorMessage: <4 keys>
    BackgroundColorEnum:
      title: background color
      type: string
      description: enumeration of allowed background colors
      enum:
      - lightgray
      - white
      - aquamarine
      - lightyellow
      - lightblue
      - '#ffe4c4'
    NewChatMessageRequest:
      required:
      - text
      type: object
      properties:
        text:
          type: string
          description: text of message
          example: Hello! 😀
        textColor:
          type: string
          description: HTML color name or RGB hex code
          example: black
          enum:
          - black
          - blue
          - darkgrey
        backgroundColor:
          $ref: '#/components/schemas/BackgroundColorEnum'
      description: |
        Object for requesting new message.
        **Text** of the message must be located in the request body because URLs are limited in size.
```

enum type definition

input DTO type definition

reference to the enum definition

```yaml
components:
  schemas: <7 keys>
  responses:
    SingleMessageResponse:
      description: response containing a single message
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ChatMessage'
      links:
        link_to_getMessage:
          operationId: getMessage
          parameters:
            id: $response.body#/id
          description: |
            The `id` value returned in the response can be used as
            the `id` parameter in `GET /message/{id}`.
```

place for defining shared responses

linking id parameter with id in response

Input in Requests

- parameters
  - in: path - in URL before ?, e.g. /message/1
  - in: query - in URL after ?, e.g. /message?id=1
  - in: header - in HTTP header, e.g. User-Agent: curl/7.81
  - in: cookie - in HTTP header Cookie, e.g. Cookie: userid=2
- in RestController's method annotate its argument with
  - in: path - @PathVariable
  - in: query - @RequestParam
  - in: header - @RequestHeader
  - in: cookie - @CookieValue
- request body
  - specifies MIME type (e.g. application/json or application/x-www-form-urlencoded) and structure of the data
  - in a RestController annotate with @RequestBody
- URLs and HTTP headers have practical size limits (~4-8 kB), request bodies do not

Operation createMessage in code

```
@Operation(
        responses = {
                @ApiResponse(responseCode = "201", ref = "#/components/responses/SingleMessageResponse"),
                @ApiResponse(responseCode = "400", description = "input data were not correct",
                        content = @Content(schema = @Schema(implementation = ErrorMessage.class))
                ),
        }
)
@PostMapping(path = "/messages")
@CrossOrigin(origins = "*")
@ResponseStatus(HttpStatus.CREATED)
public ChatMessage createMessage(@Valid @RequestBody NewChatMessageRequest r, BindingResult bindingResult,
                                 @RequestParam(required = false) String author,
                                 @RequestHeader(value = "User-agent", required = false) String userAgent) {
    // check validity of input data
    if (bindingResult.hasErrors()) {
        log.debug("binding errors found");
        for (FieldError fe : bindingResult.getFieldErrors()) {
            log.debug("FieldError: {} - {}", fe.getField(), fe.getDefaultMessage());
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, fe.getField() + " - " + fe.getDefaultMessage());
        }
    }
    // default value
    BackgroundColor bc = (r.getBackgroundColor() == null) ? BackgroundColor.LIGHTGRAY : r.getBackgroundColor();
    // create message
    StoredMessage message = chatService.createNewChatMessage(r.getText(), author, r.getTextColor(), bc.getValue());
    // return the created message
    return ChatMessage.fromStoredMessage(message);
}
```

Testing a RestController

- unit tests
  - run during “`mvn test`” phase
  - run by Maven’s sure-fire plugin, no need to declare
  - name of the class with test must end in “Test” or “Tests”
  - test RestController as a separated unit
  - service is mocked (instantiated by Mockito)
- integration tests
  - run during “`mvn verify`” phase
  - run by Maven’s fail-safe plugin, must be declared in pom.xml
  - name of the class with test must end in “IT”
  - test RestController integrated with underlying service
  - service is real (instantiated by Spring)

Unit test of getAllMessages operation

```
@WebMvcTest(ChatRestController.class)
class ChatRestControllerUnitTests {

    // injected mock implementation of Spring MVC
    @Autowired
    private MockMvc mockMvc;

    // injected mock implementation of the ChatService interface
    @MockBean
    private ChatService mockChatService;

    static StoredMessage message01 = new StoredMessage( id: "01", ZonedDateTime.now(),  text: "hello",
    static List<StoredMessage> allMessages = List.of(message01);

    @Test
    void getAllMessages() throws Exception {
        log.debug("getAllMessages");
        // define what mock service returns when called
        given(mockChatService.getAllMessages()).willReturn(allMessages);
        // call controller and check the result
        mockMvc.perform(get( urlTemplate: "/api/messages").contentType(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(jsonPath( expression: "$.[?(@.id=='01')].text").value( expectedValue: "hello"))
        ;
    }
```

MVC mock injected

service mock injected

service mock behavior defined

test checks

Unit test of incorrect input

```
@Test
void createMessageWithoutText() throws Exception {
    log.debug("createMessageWithoutText");
    // mock service is not needed in this test
    // call controller and check the result
    NewChatMessageRequest n = new NewChatMessageRequest();
    n.setText(null);
    n.setTextColor("black");
    mockMvc.perform(post( urlTemplate: "/api/messages")
                    .header( name: "User-Agent",  ...values: "007")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(n))
            )
            .andExpect(status().isBadRequest())
    ;
}
```

null text is bad input

operation should return HTTP status code
400 “bad request”

Integration Test

```java
@SpringBootTest
@AutoConfigureMockMvc
public class ChatIT {

    private static final Logger log = LoggerFactory.getLogger(ChatIT.class);

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    // real ChatService, new for each test
    @Autowired
    private ChatService chatService;

    @Test
    void testMessageIds() throws Exception {
        log.debug("Integration Test - testMessageIds");
        //create 3 more messages directly in service
        for(int i = 1; i<4; i++) {
            chatService.createNewMessage( text: "message "+i, author: null, textColor: null,  backgroundColor: null);
        }
        // get each message by calling URL and compare its content
        for (StoredMessage storedMessage : chatService.getAllMessages()) {
            // read each message by calling the RestController
            String response = mockMvc.perform(get( urlTemplate: "/api/message/{id}",storedMessage.id())
                            .contentType(MediaType.APPLICATION_JSON))
                    .andExpect(status().isOk())
                    .andReturn().getResponse().getContentAsString();
            log.debug("response: {}", response);
            ChatMessage chatMessage = objectMapper.readValue(response, ChatMessage.class);
            assertThat( reason: "id", chatMessage.id(), is(equalTo(storedMessage.id())));
            assertThat( reason: "text", chatMessage.text(), is(equalTo(storedMessage.text())));
        }
    }
```

this is different from unit tests

real instance of service is injected, not a mock

Paging

```java
public interface ChatService {

    1 implementation
    List<StoredMessage> getAllMessages();

    1 implementation
    StoredMessage createNewMessage(String text, String author, String textColor, String backgroundColor);

    1 implementation
    StoredMessage getMessage(String id);

    1 implementation
    Page<StoredMessage> getPageOfMessages(Pageable pageable);
}
```

```java
    @GetMapping(path = "/paged")
    @CrossOrigin(origins = "*")
    public Page<ChatMessage> paged(@ParameterObject Pageable pageable, HttpServletRequest req) {
        log.info("{} {} called from {}", req.getMethod(), req.getRequestURI(), req.getRemoteHost());
        return chatService.getPageOfMessages(pageable).map(ChatMessage::fromStoredMessage);
    }
```

Page

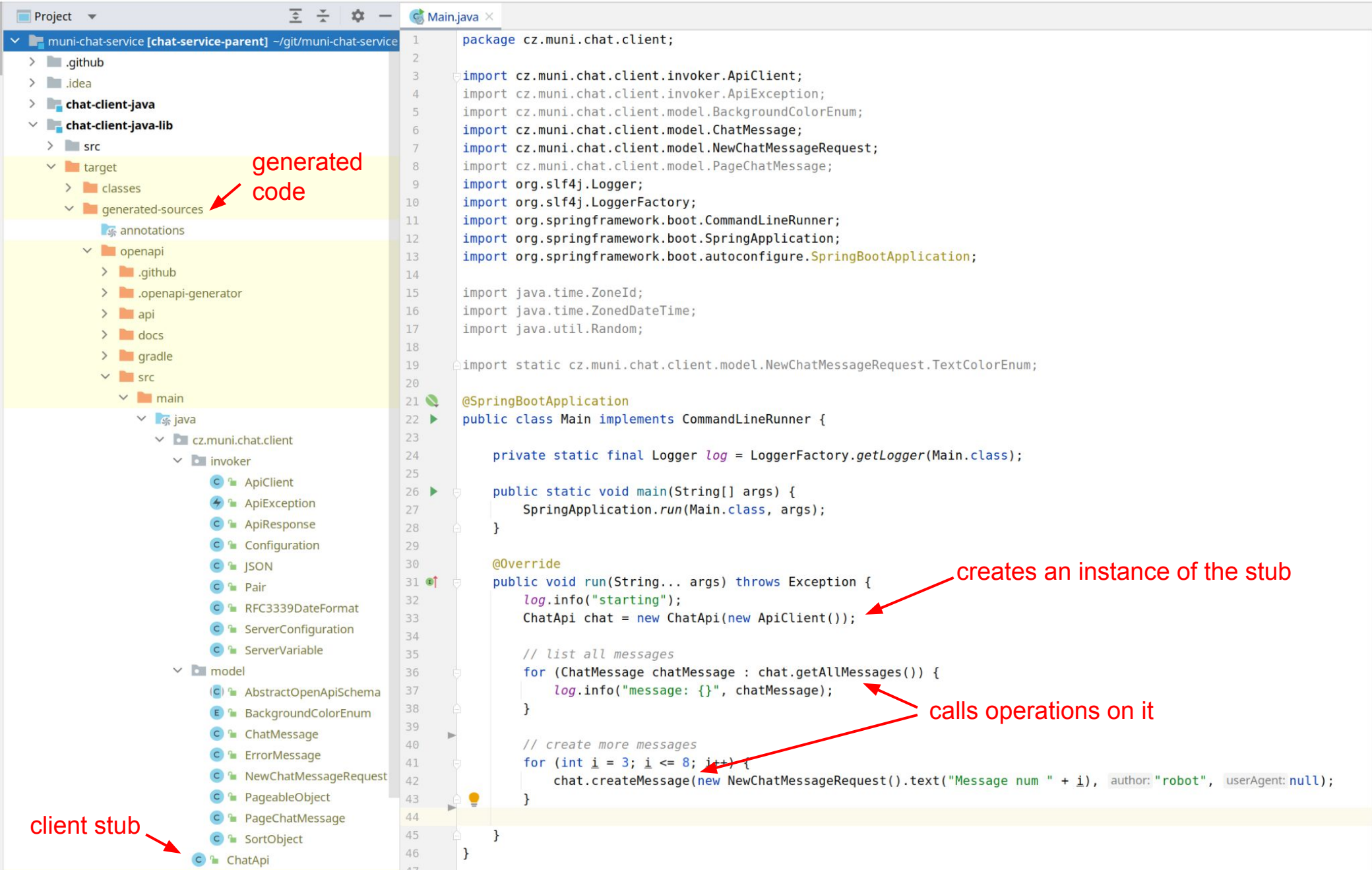
Pageable

Pageable object is propagated into Spring Data repositories and transformed into LIMIT, OFFSET, and ORDER BY keywords in SQL queries

chat-client-java-lib module

- java library - no main(String[] args) method
- packaged as JAR file
- dependency for other modules that want to call the Chat API, i.e. the chat-client-java module
- generated java code in **target/generated-sources/openapi/src/main/java**
- code generated by client stub “**java**” generator

Client calling generated stub




```java
package cz.muni.chat.client;

import cz.muni.chat.client.invoker.ApiClient;
import cz.muni.chat.client.invoker.ApiException;
import cz.muni.chat.client.model.BackgroundColorEnum;
import cz.muni.chat.client.model.ChatMessage;
import cz.muni.chat.client.model.NewChatMessageRequest;
import cz.muni.chat.client.model.PageChatMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.util.Random;

import static cz.muni.chat.client.model.NewChatMessageRequest.TextColorEnum;

@SpringBootApplication
public class Main implements CommandLineRunner {

    private static final Logger log = LoggerFactory.getLogger(Main.class);

    public static void main(String[] args) {
        SpringApplication.run(Main.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        log.info("starting");
        ChatApi chat = new ChatApi(new ApiClient());

        // list all messages
        for (ChatMessage chatMessage : chat.getAllMessages()) {
            log.info("message: {}", chatMessage);
        }

        // create more messages
        for (int i = 3; i <= 8; i++) {
            chat.createMessage(new NewChatMessageRequest().text("Message num " + i), author: "robot", userAgent: null);
        }

    }
}
```

Maven Plugin for OpenAPI Generator

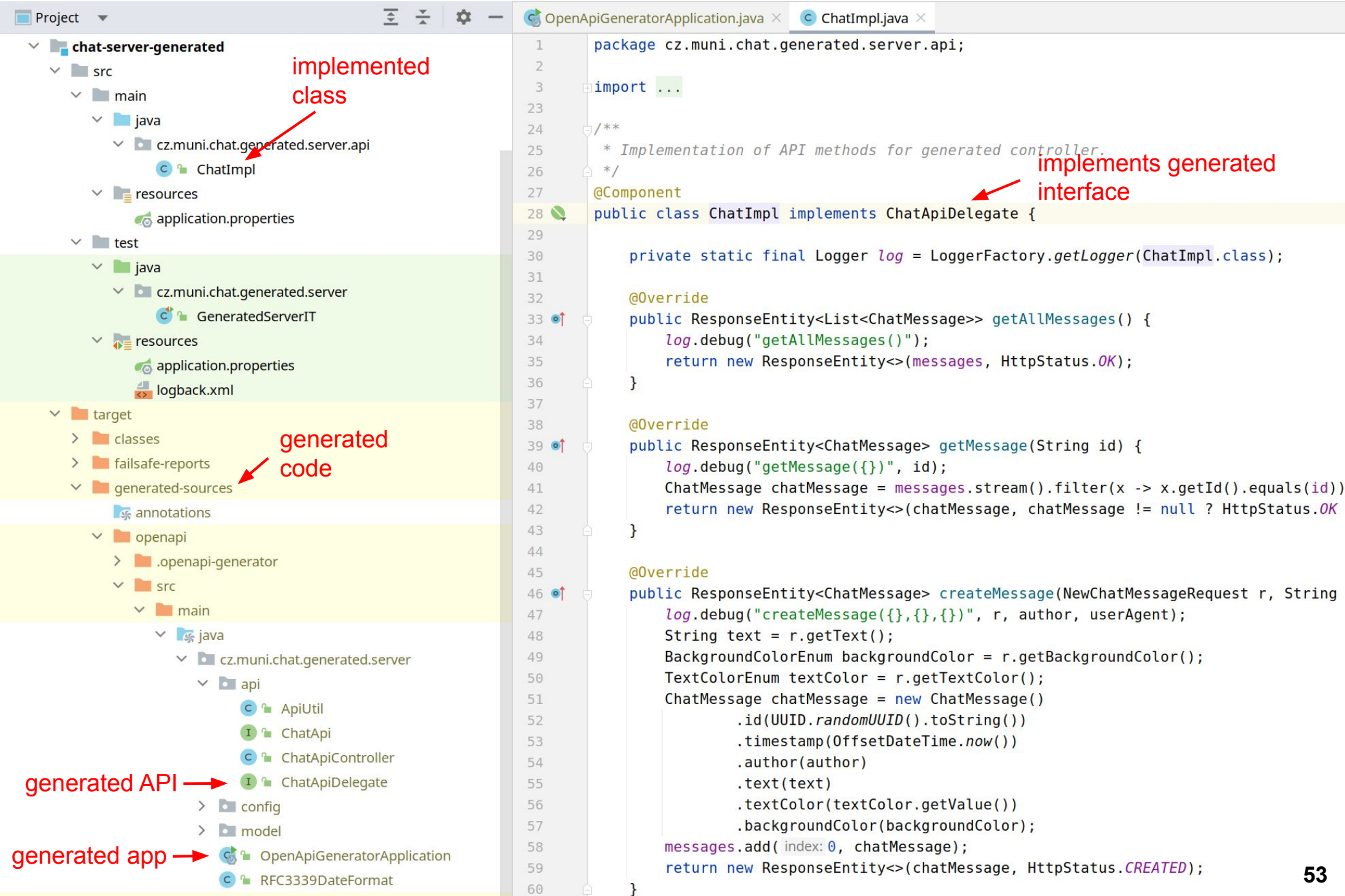
```
<plugins>
    <plugin>
        <groupId>org.openapitools</groupId>
        <artifactId>openapi-generator-maven-plugin</artifactId>
        <executions>
            <execution>
                <goals>
                    <goal>generate</goal>
                </goals>
                <configuration>
                    <!-- see https://github.com/OpenAPITools/openapi-generator/blob/master/modules/openapi-generator-maven-
                    <inputSpec>${project.basedir}/../openapi.yaml</inputSpec>
                    <generatorName>java</generatorName>
                    <apiPackage>cz.muni.chat.client</apiPackage>
                    <modelPackage>cz.muni.chat.client.model</modelPackage>
                    <invokerPackage>cz.muni.chat.client.invoker</invokerPackage>
                    <verbose>false</verbose>
                    <generateApiTests>false</generateApiTests>
                    <generateModelTests>false</generateModelTests>
                    <generateApiDocumentation>true</generateApiDocumentation>
                    <generateModelDocumentation>true</generateModelDocumentation>
                    <configOptions>
                        <annotationLibrary>none</annotationLibrary>
                        <!-- see https://openapi-generator.tech/docs/generators/java/ -->
                        <library>native</library>
                        <hideGenerationTimestamp>true</hideGenerationTimestamp>
                    </configOptions>
                </configuration>
            </execution>
        </executions>
    </plugin>
</plugins>
```

location of OpenAPI document

client stub generator “java”

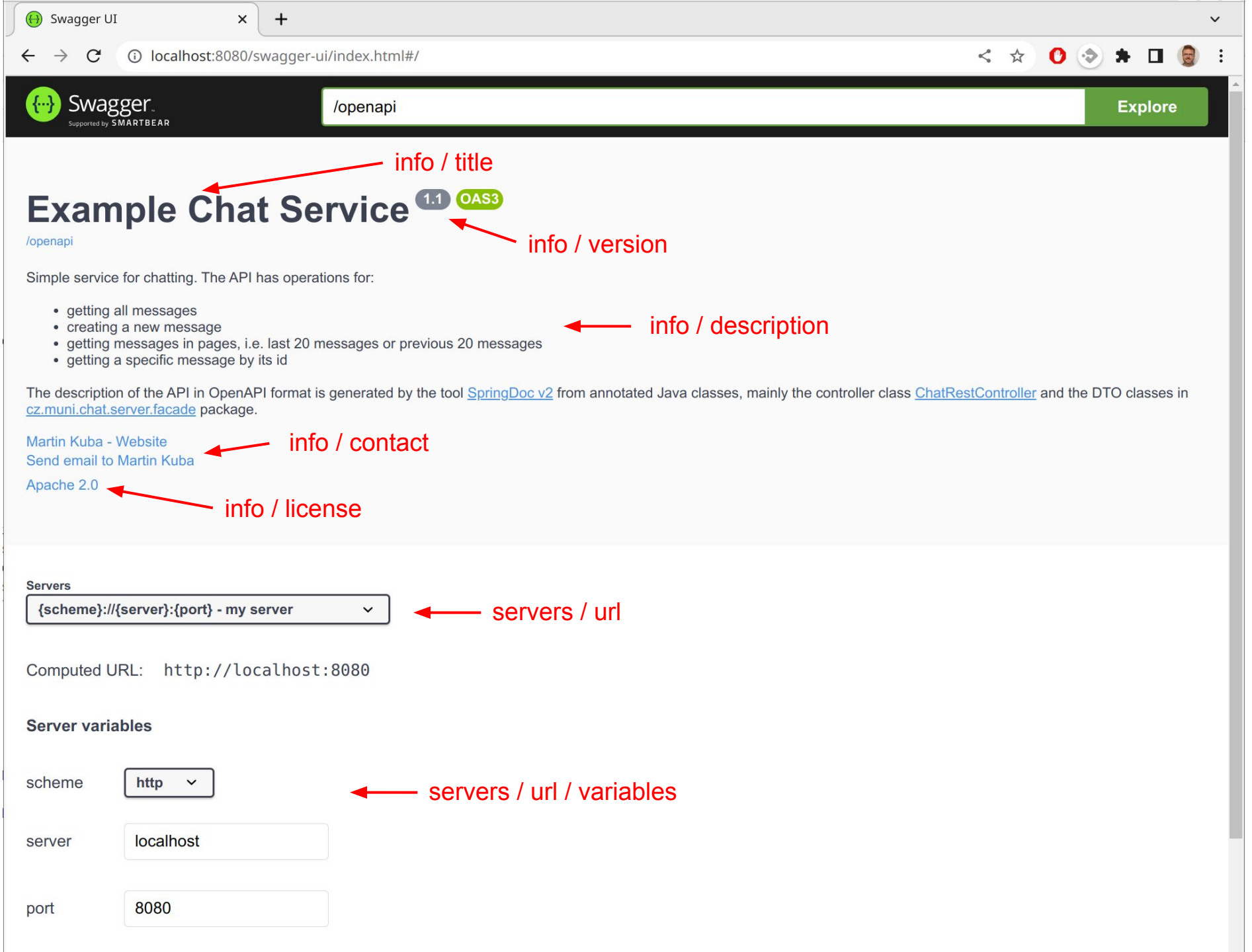
chat-server-generated module

- API-first approach example
- produces an executable JAR file
- generated java code again in **target/generated-sources/openapi/src/main/java**
- code generated by server skeleton “**spring**” generator
- implemented just a single class **ChatImpl** that implements generated ChatApiDelegate interface


Project
chat-server-generated
src
main
java
cz.muni.chat.generated.server.api
ChatImpl
resources
application.properties
test
java
cz.muni.chat.generated.server
GeneratedServerIT
resources
application.properties
logback.xml
target
classes
failsafe-reports
generated-sources
annotations
openapi
.openapi-generator
src
main
java
cz.muni.chat.generated.server
api
ApiUtil
ChatApi
ChatApiController
ChatApiDelegate
config
model
OpenApiGeneratorApplication
RFC3339DateFormat
implemented class
generated code
generated API
generated app
OpenApiGeneratorApplication.java
ChatImpl.java
implements generated interface
1 package cz.muni.chat.generated.server.api;
3 import ...
24 /**
25 * Implementation of API methods for generated controller.
26 */
27 @Component
28 public class ChatImpl implements ChatApiDelegate {
30 private static final Logger log = LoggerFactory.getLogger(ChatImpl.class);
32 @Override
33 public ResponseEntity<List<ChatMessage>> getAllMessages() {
34 log.debug("getAllMessages()");
35 return new ResponseEntity<>(messages, HttpStatus.OK);
36 }
38 @Override
39 public ResponseEntity<ChatMessage> getMessage(String id) {
40 log.debug("getMessage({})", id);
41 ChatMessage chatMessage = messages.stream().filter(x -> x.getId().equals(id))
42 return new ResponseEntity<>(chatMessage, chatMessage != null ? HttpStatus.OK
43 }
45 @Override
46 public ResponseEntity<ChatMessage> createMessage(NewChatMessageRequest r, String
47 log.debug("createMessage({},{},{})", r, author, userAgent);
48 String text = r.getText();
49 BackgroundColorEnum backgroundColor = r.getBackgroundColor();
50 TextColorEnum textColor = r.getTextColor();
51 ChatMessage chatMessage = new ChatMessage()
52 .id(UUID.randomUUID().toString())
53 .timestamp(OffsetDateTime.now())
54 .author(author)
55 .text(text)
56 .textColor(textColor.getValue())
57 .backgroundColor(backgroundColor);
58 messages.add(index: 0, chatMessage);
59 return new ResponseEntity<>(chatMessage, HttpStatus.CREATED);
60 }

Info, Servers,Tags sections in openapi.yaml

```yaml
openapi: 3.0.1
info:
  title: Example Chat Service
  description: Simple se…) package.
  contact:
    name: Martin Kuba
    url: https://www.muni.cz/lide/3988-martin-kuba
    email: makub@ics.muni.cz
  license:
    name: Apache 2.0
    url: https://www.apache.org/licenses/LICENSE-2.0.html
  version: "1.1"
servers:
- url: "{scheme}://{server}:{port}"
  description: my server
  variables:
    scheme:
      default: http
      enum: [ http, https ]
    server:
      default: localhost
    port:
      default: "8080"
tags:
- name: Chat
  description: microservice for chat
paths: <3 keys>
components:
  schemas: <7 keys>
  responses:
```


Swagger UI
localhost:8080/swagger-ui/index.html#/
Swagger
Supported by SMARTBEAR
/openapi
Explore
info / title
Example Chat Service
1.1
OAS3
info / version
/openapi
Simple service for chatting. The API has operations for:
getting all messages
creating a new message
getting messages in pages, i.e. last 20 messages or previous 20 messages
getting a specific message by its id
info / description
The description of the API in OpenAPI format is generated by the tool SpringDoc v2 from annotated Java classes, mainly the controller class ChatRestController and the DTO classes in cz.muni.chat.server.facade package.
Martin Kuba - Website
Send email to Martin Kuba
info / contact
Apache 2.0
info / license
Servers
{scheme}://{server}:{port} - my server
servers / url
Computed URL: http://localhost:8080
Server variables
scheme
http
server
localhost
port
8080
servers / url / variables

Description of Authentication

```yaml
security:
  - BasicAuth: [ ]
  - BearerAuth: [ ]
  - OAuth2: [ "read", "write" ]

components:
  securitySchemes:
    BasicAuth:
      description: "HTTP Basic authentication with username and password"
      type: http
      scheme: basic
    BearerAuth:
      description: "OAuth2 Resource Server authentication using bearer access token"
      type: http
      scheme: bearer
      bearerFormat: "JWT"
    OAuth2:
      description: "OAuth2 more specific description"
      type: oauth2
      flows:
        authorizationCode:
          authorizationUrl: "https://as.example.org/authorize"
          tokenUrl: "https://as.example.org/token"
          scopes:
            read: "allows to read"
            write: "allows to write"
```

Python client generation

- use OpenAPI Generator with “python-prior” generator
- specify packageName for Python package

```
#!/bin/bash

GENERATOR_VERSION=6.3.0
if [ ! -f  "openapi-generator-cli-$GENERATOR_VERSION.jar" ] ; then
  wget https://repo1.maven.org/maven2/org/openapitools/openapi-generator-cli/$GENERATOR_VERSION/openapi-generator-cli-$GENERATOR_VERSION.jar
fi

rm -rf chat_openapi
# see https://openapi-generator.tech/docs/usage#generate
# and https://openapi-generator.tech/docs/generators/python-prior
java \
 --add-opens java.base/java.util=ALL-UNNAMED \
 --add-opens java.base/java.lang=ALL-UNNAMED \
 -DapiDocs=true \
 -DapiTests=false \
 -DmodelDocs=true \
 -DmodelTests=false \
 -jar openapi-generator-cli-$GENERATOR_VERSION.jar generate \
 --generator-name python-prior \
 --input-spec ../openapi.yaml \
 --model-package model \
 --additional-properties=generateSourceCodeOnly=true,packageName=chat_openapi
```

Python client code

```python
#!/usr/bin/env python3
from typing import List

from chat_openapi.exceptions import ApiException
from chat_openapi.configuration import Configuration
from chat_openapi.api_client import ApiClient
from chat_openapi.api.chat_api import ChatApi
from chat_openapi.model.chat_message import ChatMessage
from chat_openapi.model.new_chat_message_request import NewChatMessageRequest
from chat_openapi.model.background_color_enum import BackgroundColorEnum


configuration = Configuration(host="http://localhost:8080")
configuration.debug = False
api_client: ApiClient = ApiClient(configuration)
api_client.user_agent = "MyChat(Python) 1.0"
chat: ChatApi = ChatApi(api_client)

try:
    # get all messages
    messages: List[ChatMessage] = chat.get_all_messages()
    for msg in messages:
        print("(%s)" % msg.author, msg.text)

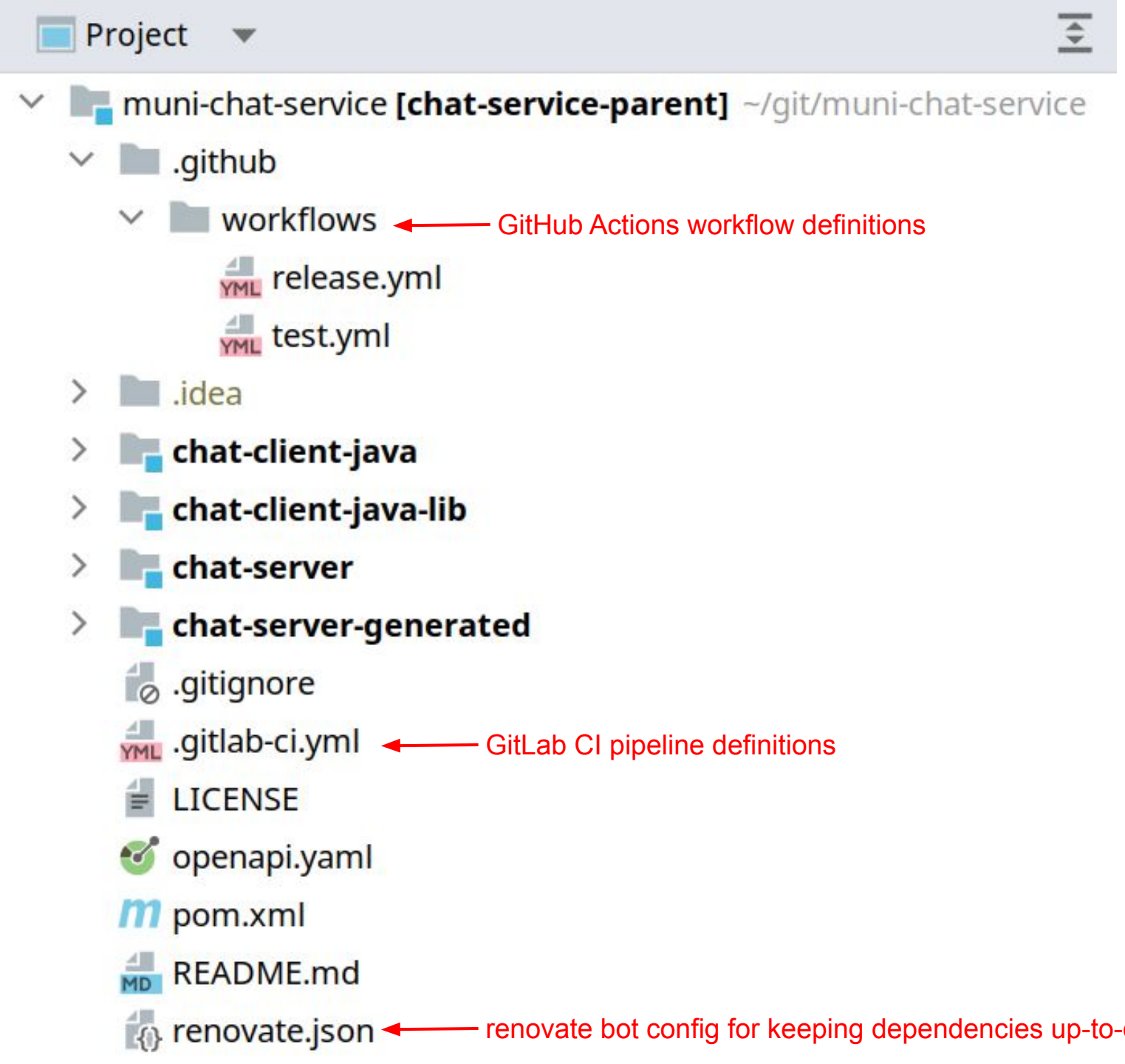
    # create a new message
    msg_req = NewChatMessageRequest(text="Hello!", text_color="black", background_color=BackgroundColorEnum("lightgray"))
    msg: ChatMessage = chat.create_message(msg_req, author="Joe")
    print("(%s)" % msg.author, msg.text)
except ApiException as e:
    print("Exception: %s\n" % e)
```

creates instance of the generated ChatApi class

calls getAllMessages operation, note the change from camelCase to snake_case naming

calls createMessage operation

The end


Project
muni-chat-service [chat-service-parent] ~/git/muni-chat-service
.github
workflows
GitHub Actions workflow definitions
release.yml
test.yml
.idea
chat-client-java
chat-client-java-lib
chat-server
chat-server-generated
.gitignore
.gitlab-ci.yml
GitLab CI pipeline definitions
LICENSE
openapi.yaml
pom.xml
README.md
renovate.json
renovate bot config for keeping dependencies up-to-date